

Technical University of Berlin

Distributed Artificial Intelligence Laboratory (DAI-LABOR)

Applications of Robotics and Autonomous Systems



Prediction, Perception and Advanced AD Capabilities

System Design Specifications

Working Group 2

Alexandros Nikolaou



Sharang Kaul



Sumit Patidar



Jhorman Perez



Stoyan Karastoyanov



Janis Freund



March 10, 2021

Contents

1	Introduction	3
1.1	Objectives	3
1.2	Use cases/Scenarios	4
1.3	General Constraints	5
2	State-of-the-Art	6
2.1	Sensing	7
2.2	Localization and Mapping	8
2.3	Perception Algorithms	10
2.3.1	Image-based object detection	10
2.3.2	Semantic Segmentation	10
2.3.3	3D Object Detection	11
2.3.4	Road and Lane Detection	12
2.3.5	Object Tracking	13
2.4	Prediction Algorithms	13
2.4.1	Physics-based Models	14
2.4.2	Maneuver recognition	14
2.5	Planning and Decision Making	15
2.5.1	Global Planning	15
2.5.2	Local Planning	15
2.5.3	Decision Making	16
2.6	Acting	17
2.6.1	Control	17
2.7	End to End Learning	18
2.8	Advanced Capabilities	19
2.8.1	Parking	19
3	System Requirements, Architecture and Specifications	20
3.1	Requirements	20
3.2	System Architecture	20
3.3	System Specifications	21
4	Timeline	25

1 Introduction

For thousands of years, horses were the preferred mode of movement and during the 19th and 20th centuries were still regarded as ruling the urban life and rural culture in terms of transportation [1]. From then on followed the transition to motorized cars facilitated by the adoption of three fossil-fuel technologies. Steam engines were used for long-distance haulage, coal-fired electricity made the horse redundant for public transit and the combustion engine replaced the horse as the preferred mode of transportation of individuals and most goods. In current times, driven by both technological and socioeconomic factors, transportation has evolved to a point where a further transition to driverless vehicles seems inevitable.

In this report, we aim to illustrate the functional and technical specifications of an autonomous vehicle project provided by the DAI-LABOR at the Technical University of Berlin. The main objective of this project is to introduce the future of robotic autonomous vehicles and their applications in a simple manner. To that end, a small scale car, a track with lanes and a set of traffic signs will be provided. The goal at the end of the projects is for this car to traverse the track with lanes, properly follow the traffic signs and park into a designated spot, all of that - autonomously. This goal can be achieved within a simulation or if circumstances permit, in the DAI-LABOR physical workplace. Since autonomous driving is a vast field involving numerous subdomains the whole project is divided into working groups. Our group - number 2 focuses on the prediction, perception and decision making capabilities of the car. These capabilities, in turn, open up the possibilities for an autonomous vehicle to follow traffic rules, drive inside the lanes, avoid static and dynamic obstacles, and handle complex manoeuvring with ease. The report describes the project objectives, use cases, challenges, state-of-art techniques, system design specifications and provides a timeline to track the milestones.

1.1 Objectives

The overall objective of this project is to develop the necessary functionalities of a small scale car for autonomous driving relying on custom perception and prediction engines plus some advanced autonomous driving capabilities - traffic jam assist and parking.

As inputs to our work, we are given a set of ready-to-use components of the self-driving functionality, that include a low-level tuned PID motion planner, a global A* planner, sensor fusion and localization data (EKF, SLAM) and a map with lanes for testing purposes. In addition, we have access to a labelled dataset of images for the object recognition tasks. Next, the specific objectives:

- Define work division and assign project roles.
- Installation and understanding of existing code base.
- Set up the self-driving simulation environment for the 4WD car.
- Identify the possible scenarios the robot can be exposed to.
- Implement the perception module: object recognition, obstacle detection and Local Dynamic Map (LDM) generation.

- Implement the prediction module: future behaviours of the traffic entities and road curves estimation.
- Implement the local navigation planning module for navigation, based on the perception and prediction outputs.
- **Optional objectives on existing code base**
 - Tune the localization package already provided to get a better performance.
 - Configure a SLAM package for the 4WD car different from one by default (Hector SLAM).
 - Tune the default controller for motion planning (PID).
 - Implement an improved version of the global planning algorithm (A*).

1.2 Use cases/Scenarios

Given a destination point, the car should come up with a global path start navigating in the correct direction. Once the vehicle starts heading the trajectory, the following scenarios may take place:

1. The car is trying to go from A to B and it detects a speed bump on the road, so it should decelerate.
2. The robot finds itself in an intersection and it may need to decide whether to go straight or turn left or right, depending on the global planning.
3. The car needs to interrupt its goal execution because an alert of low battery is triggered, so it has to find a charging station. After, it can continue with the original target.
4. The lanes are not visible anymore in some sections of the road and the car loses this reference to keep within the track; it would have to rely on other information to not get out of the safe path or cause an accident.
5. The car approaches a construction area or slipperiness stretch and it may need to slow down and replan the local navigation path. There might be or not signs indicating about these incidents.
6. Wherever there is a stop sign the car has to look around the intersection to see if it's car-free to take the turn or to simply go ahead.
7. It turns out that there are people trying to cross the street unexpectedly where they shouldn't or other cars appearing on the way of our robot; it has to handle the situation by triggering an emergency brake and then wait after there are no obstacles to go back to drive normally.
8. When a crosswalk is detected in the vicinity of the car, it needs to start braking and wait for any people that try to cross the street.
9. There is a car parked unexpectedly in the middle of the road, having the robot navigate around it, surpassing it and resuming the trajectory.

10. If another moving car is in front of the robot, the latter regulates its speed to keep a safe distance to each other.
11. The robot can't find any available parking spot (explicitly indicated). It has to work around this situation and try to park somewhere else where there's an empty space.

1.3 General Constraints

This project makes these assumptions according to the given scenario.

- The number of the robots will be limited.
- Only the following set of signs is recognized: turn right/left, construction area, slipperiness, stop, parking place.
- The shape of all objects is either cubic or human.
- The car just has to stay in the lane without having to drive at the correct side.
- There is a general speed limit which will not be exceeded by the car.
- Charging stations are always indicated by a right arrow in front of them.
- Intersections consist of a maximum of three converging lanes.

2 State-of-the-Art

The autonomous vehicle system can be divided into four main categories:

- **Sensing:** The vehicle is sensing the world using many different sensors mounted on the vehicle. These are hardware components that gather data about the environment.
- **Perception and Prediction:** The information from the sensors is processed in perception and prediction blocks whose components combine sensor data into meaningful information.
- **Planning and Decision Making:** Planning subsystem uses the output from perception and prediction block for behavior planning and for both short and long-range path plan.
- **Control:** Control module ensures that the vehicle follows the path provided by the planning subsystem and sends control commands to the vehicle.

The complete integration of on-board hardware, on-board software components and off-board hardware/software components is described as follows:

On-board hardware:

- **Camera/Optics:** Collects optical images to be interpreted by advanced AI and analytics.
- **Radar:** Determine speed and distance of object using electromagnetic waves.
- **LiDAR:** High resolution sensing using light beams to estimate distance from obstacles.
- **Ultrasonic Sensors:** Short distance object recognition (e.g. parking).
- **Odometry Sensors:** Measure wheel speed to predict vehicle travel and compliment localization.
- **GPS:** Localization of the vehicle using satellite triangulation.
- **Actuators:** Translation of electronic signals to mechanical actions.
- **V2V/V2I:** Communications with vehicles and infrastructure over short range.
- **System on a Chip (SOC):** High performance energy-efficient computer hardware.

On-board software components :

- **High Definition On-Board Maps:** Precise localization information about roads, infrastructure and environment.
- **Localization and Mapping:** Data fusion for vehicle localization and environment analysis.

- **Perception and Object Analysis Algorithms:** Detection and classification of objects and obstacles.
- **Prediction:** Foresight of movement and actions by vehicles, pedestrians and other moving objects.
- **Decision Making:** Planning of vehicle route, maneuvers, acceleration, steering and braking.
- **Vehicle Operating Systems:** Operating systems running algorithms in real time.
- **Supervision Platform:** Analytics to monitor the AV system operation, detecting and correcting faults.

Off-board software/hardware components :

- **Data Center:** Storage and processing of hot and cold vehicle data
- **AV Cloud Operation:** Learning, adopting and updating HD maps and algorithms

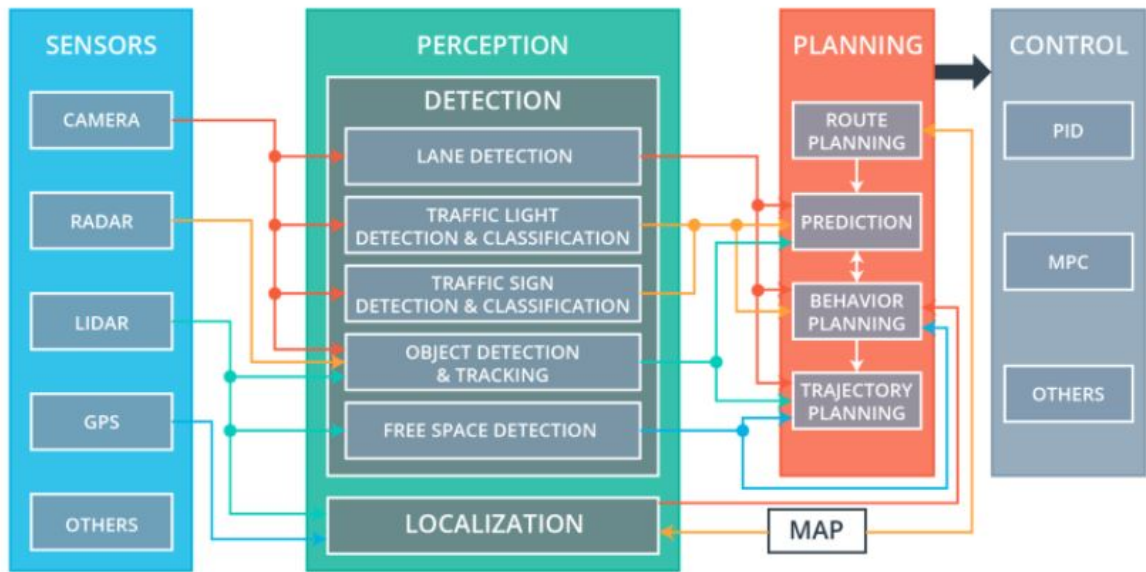


Figure 1: Overview of a Self-driving Car system [2]

2.1 Sensing

Sensor fusion is an approach for combining data delivered from disparate sources such that the coherent information is created. The resulting information is more certain than it would be possible when these sources were used individually. This is especially important when different kinds of information are combined. For example, on the autonomous vehicle, it is important to have a camera in order to clone a human vision, since they are able to detect RGB information and have high resolution. However it has some issues

under sunlight conditions and contrast and depth tasks are rather challenging for these sensors. The information of the obstacle distance, on the other hand, will be the best gained through the sensors like LiDAR or radar. For that reason, sensor fusion of Camera with LIDAR or Radar data is very important since they are complementary. On the other hand, combining information from LiDAR and Radar will provide more certain information about the distance of the obstacle ahead of the vehicle or general distance of the objects in the environment.

The following technical factors affect the choice of sensors:

- The scanning range, determining the amount of time you have to react to an object that is being sensed
- Resolution, determining how much detail the sensor can give you.
- Field of view or the angular resolution, determining how many sensors you would need to cover the area you want to perceive.
- Ability to distinguish between multiple static and moving objects in 3D, determining the number of objects you can track.
- Refresh rate, determining how frequently the information from the sensor is updated.
- General reliability and accuracy in different environmental conditions.
- Cost, size and software compatibility.
- Amount of data generated.

The best choice of sensors for the experiment would be LiDAR, Cameras and Radar. But according to the application, the fusion of sensors on the board of the vehicle will change accordingly[3].

It is noteworthy to mention that one of the biggest autonomous vehicle makers, Tesla, does not use LIDARs and mainly relies on the cameras. A similar approach is being used by Wayve which deployed end-to-end deep learning autonomous driving. On the other hand, Volvo-Uber and Waymo have a top-mounted 360 degree Lidar to detect road objects.

2.2 Localization and Mapping

Localization is the task of finding ego-position relative to a reference frame in an environment, and it is fundamental to any mobile robot. It is especially crucial for automated driving systems; the vehicle must use the correct lane and position itself in it accurately. Furthermore, localization is an elemental requirement for global navigation. The remainder of this section details the three most common approaches that use solely on-board sensors: Global Positioning System and Inertial Measurement Unit (GPS-IMU) fusion, Simultaneous Localization And Mapping (SLAM), and state-of-the-art a priori map-based localization. A comparison of localization methods is given in Table 1.

Table 1. Comparison of Localization Techniques

Methods	Robustness	Cost	Accuracy	Size	Computational Requirements
Absolute Positioning Sensors	Low	Low	Low	Small	Lowest
Odometry/Dead Reckoning	Low	Low	Low	Smallest	Low
GPS-IMU Fusion	Medium	Medium	Low	Small	Low
SLAM	Medium-High	Medium	High	Large	Very-High
Priori Map-based Localization					
Landmark Based	High	Medium	High	Large	Medium
Point-cloud matching	Highest	Highest	Highest	Largest	High

The best possible methods available for localization and mapping relates to the idea of map-based localization. The core idea of a priori map-based localization techniques is matching: localization is achieved through the comparison of online readings to the information on a detailed pre-built map and finding the location of the best possible match.

SLAM: SLAM problem has two formulations: Full SLAM where we estimate the whole trajectory of the vehicle and the map, given the control inputs and the measurements. And the online SLAM which estimates the current position of the vehicle based on the last sensor information. Moreover, the estimation can be separated into two categories: Filter-based and Optimization-based SLAM [4]. EKF SLAM and Particle Filter SLAM are the two main methods used in the Filter-based category. Whereas Graph-based SLAM is leading the way in the Optimization-based SLAM.

More recent developments use vision-based SLAM approaches with ORBSLAM2 [5] and GCNv2 [6] being the most efficient in the field.

hector_mapping is a SLAM approach that can be used without odometry as well as on platforms that exhibit roll/pitch motion (of the sensor, the platform or both).

Odometry (and control input) may be unreliable in some scenarios (e.g. in search and rescue environments). Thus, hector only relies on LiDAR measurements.

It leverages the high update rate of modern LiDAR systems and provides 2D pose estimates at scan rate of the sensors. While the system does not provide explicit loop closing ability, it is sufficiently accurate for many real world scenarios.

SLAM will be constantly combined with the dynamic map for a better accuracy.

Pose Estimation: For localizing the robot we will use Extended Kalman Filter (EKF) from IMU measurements and 2d pose error from laser scanner. EKF is an extension of Kalman Filter to handle the non-linearities. For the EKF we use the **robot_localization** package. We change the covariance values to alter how precise the localization is. Smaller covariance means more precision.

An alternative to EKF is Particle filters or Monte-Carlo localization. Particle filtering uses a set of particles (also called samples) to represent the posterior distribution of some stochastic process given noisy and/or partial observations. The state-space model can be

nonlinear and the initial state and noise distributions can take any form required. Particle filter techniques provide a well-established methodology for generating samples from the required distribution without requiring assumptions about the state-space model or the state distributions. However, these methods do not perform well when applied to very high-dimensional systems.

2.3 Perception Algorithms

Perceiving the surrounding environment and extracting information which may be critical for safe navigation is a major objective for automated driving systems. A variety of tasks, using different sensing modalities, fall under the category of perception. Building on decades of computer vision research, cameras are the most commonly used sensor for perception, with 3D vision becoming a strong alternative/supplement. The remainder of this section is divided into core perception tasks. We discuss image-based object detection, semantic segmentation, 3D object detection, road and lane detection and object tracking.

2.3.1 Image-based object detection

While state-of-the-art methods all rely on Deep Convolutional Neural Networks (DCNNs), there currently exist a clear distinction between them:

- Single stage detection frameworks use a single network to produce object detection locations and class prediction simultaneously. For our simulation, we can use algorithms like **You Only Look Once (YOLO)** or **Single Shot Detector (SSD)** which use standard DCNNs architectures such as **VGG** to achieve competitive results
- Region proposal detection frameworks use two distinct stages, where general regions of interest are first proposed, then categorized by separate classifier networks.

Considering both accuracy and computational cost is essential for detection in automated driving; the detection needs to be reliable, but also operate better than real-time, to allow as much time as possible for the planning and control modules to react to those objects. As such, single stage detectors are often the detection algorithms of choice for automated driving. However, region proposal networks (RPN), used in two-stage detection frameworks, have proven to be unmatched in terms of object recognition and localization accuracy, and computational cost has improved greatly in recent years. They are also better suited for other tasks related to detection, such as semantic segmentation as discussed in upcoming section. Through transfer learning, RPNs achieving multiple perception tasks simultaneously are becoming increasingly feasible for online applications. RPNs can replace single stage detection networks for ADS applications in the near future.

But for this particular type of experimentation and simulation, our team would like to pursue and implement single-stage detection frameworks considering both the computational cost and the accuracy required[7].

2.3.2 Semantic Segmentation

Image segmentation consists of classifying each pixel of an image with a class label. This task is of particular importance to driving automation as some objects of interest are

poorly defined by bounding boxes, in particular roads, traffic lines, sidewalks and buildings. As opposed to semantic segmentation, which labels pixels based on a class, instance segmentation algorithms further separates instances of the same class, which is important in the context of driving automation. In other words, objects which may have different trajectories and behaviors must be differentiated from each other. Data sets like Common Objects in Context (COCO) or any other given labelled data set can be used to train the instance segmentation algorithms like Mask R-CNN.

The multi-task R-CNN network can achieve accurate bounding box estimation and instance segmentation simultaneously and can also be generalized to other tasks like pedestrian pose estimation with minimal domain knowledge. Unlike Mask-RCNNs architecture which is more similar to those used for object detection through its use of region proposal networks, segmentation networks usually employ a combination of convolutions for feature extraction. Those are followed by deconvolutions, also called transposed convolutions, to obtain pixel resolution labels. The idea of sparse convolutions was used to develop DeepLab; the most recent version being the current state-of-the-art for object segmentation[7].

For our simulated environment, Mask R-CNN can be of prime importance to separate instances of the same class.

2.3.3 3D Object Detection

Convolutional Neural Network for 2D object detection and classification is mature. But, 3D object detection for automated driving poses at least two unique challenges:

- Unlike RGB images, LiDAR point clouds are 3D and unstructured.
- 3D detection for automated driving must be fast ($< 100\text{ms}$).

Several 3D detection methods have tackled the first problem by discretizing the LiDAR point cloud into a 3D voxel grid and applying 3D convolutions. However, 3D convolution suffers from greater computational cost and thus higher latency than 2D convolution. Alternatively, the point cloud can be projected to a 2D image in the top-down Bird Eye View (BEV) or the LiDAR's native Range View (RV). The advantage is that the projected image can be efficiently processed with faster 2D convolutions, yielding lower latency.

LIDAR + RGB fusion improve 3D detection performance, particularly for smaller objects (e.g., pedestrians) or at long range ($> 50\text{m}$ - 70m) where LiDAR data is often sparse. A few fusion approaches are summarized below:

- Proposal based methods generate object proposals in RGB (e.g., F-Pointnet) or BEV (e.g., MV3D)
- Dense fusion methods fuse LIDAR and RGB features directly into a common projection and often at various levels of resolution.

There are trade-offs between BEV and RV projections. BEV preserves metric space, keeping object size consistent with respect to range. In contrast, RV suffers from scale variation with respect to range and occlusion. As a result, BEV detectors (e.g., PointPillars) achieve superior performance to RV (e.g., LaserNet) on small datasets (e.g., 7.5k

frames) with similar latency (e.g., 16ms for PointPillars vs 12ms for LaserNet). However, RV performance is at parity with BEV on larger (e.g., 1.2M frames) datasets. Despite this drawback, dense feature fusion is faster in RV than BEV. LaserNet++ reports impressive latency (38ms) and better performance than dense BEV fusion detectors (e.g., ContFuse at 60ms). New LIDAR + RGB fusion architectures may find ways to move between projections, taking advantage of the benefits of each one. Several algorithms have been produced recently, with accuracy constantly improving as shown in Table 2.

Table 2. Average Precision (AP) in % on the KITTI 3D object detection test set car class, ordered based on moderate category accuracy (point cloud data)[8].

Algorithm	T(sec)	Easy	Moderate	Hard
PointRCNN	0.10	85.9	75.8	68.3
PointPillars	0.02	79.1	75.0	68.3
SECOND	0.04	83.1	73.7	66.2
IPOD	0.20	82.1	72.6	66.3
F-PointNet	0.17	81.2	70.4	62.2
VoxelNet (LiDAR)	0.23	77.5	65.1	57.7
MV3D (LiDAR)	0.24	66.8	52.8	51.3

2.3.4 Road and Lane Detection

For automated driving purposes, lane detection is critical in identifying and ensuring safe driving practices. An on-board system with this capability can alert the driver in the event of an unsafe lane change. Lane detection is also instrumental in understanding road layout and surrounding scene geometry for proper route planning.

Feature-Based Approach: In the feature-based approach, low-level edge features are typically used for lane boundary detection. With more complex scenery (for example, the appearance of buildings in an urban street view, or shadows cast by nearby structures or plant life), low-level features no longer stand the test.

Model-Based Approach: In addition to simple feature-based lane detection, we have also relied on the common “rules” of lane configuration we have come to expect, such as curved or straight lines remaining in parallel. Barring a few exceptions, lane lines are generally parallel. However, in an image, parallel lines meet at a central point, called a vanishing point (VP). To filter straight lines from a noisy edge map, least squares or **Linear Hough Transform** has been widely used. Although such methods are intuitive and efficient, they cannot distinguish between non-lane edges or lines in the images (such as road curbs, multiple lane lines, shadows), nor can they handle false negatives caused by erased or inconsistent lane boundary markings. Data-driven approaches can better handle these situations.

Data-Driven Approach: Deep learning learns the patterns from a bulk of images with labelled or annotated ground truth in features that are cascaded in the hidden layers of the architecture. This is referred to as supervised classification of lane markings versus background. Learning the position of the vanishing point with or without ground truth helps to generate a semblance of lane lines, even with the absence of actual lines on the road or with lines obstructed by traffic[9]. Because learning from data requires a large number of annotated images, several annotated datasets are now available for training

DL-based networks for lane detection; these include **CULane**, **TuSimple**, **CalTech**, and **BDD**. To further improve the results, DL can also combine information about the lane region and the lane boundaries.

To our knowledge, VPGNet [10] approach which combines the advantages of deep learning methods with prior knowledge about the geometry of the road (parallel lines, distances etc) provides promising results for road and lane detection.

2.3.5 Object Tracking

Object tracking means estimating the state of the target object present in the scene from previous information. On a high level of abstraction, there are mainly two levels of object tracking.

- Single Object Tracking(SOT)
- Multiple Object Tracking(MOT)

Object tracking is not limited to 2D sequence data and can be applied to 3D domains as well. The strength of Deep Neural Networks (DNN) resides in their ability to learn rich representations and to extract complex and abstract features from their input.

Multiple Object Tracking (MOT), also called Multi-Target Tracking (MTT), is a computer vision task that aims to analyse videos to identify and track objects belonging to one or more categories, such as pedestrians, cars, animals and inanimate objects, without any prior knowledge about the appearance and number of targets[11]. While in Single Object Tracking (SOT) the appearance of the target is known a priori, in MOT a detection step is necessary to identify the targets that can leave or enter the scene. The main difficulty in tracking multiple targets simultaneously stems from the various occlusions and interactions between objects that can sometimes also have a similar appearance. Thus, merely applying SOT models directly to solve MOT leads to poor results, often incurring in target drift and numerous ID switch errors, as such models usually struggle in distinguishing between similar-looking intra-class objects.

In recent years, due to the exponential rise in the research of deep learning methods, there have been tremendous gains in accuracy and performance of the detection and tracking approaches. Most of the state of the art tracking approaches follow the ‘**Tracking by Detection**’ scheme where they first find objects in the scene and then find the corresponding tracklets (position of it in the next frame) of the objects. The most popular Tracking by detector method is Tractor [12]. Today the detectors are performing exceptionally well and can scale to different scene adaptations. Consequently, it has led to the standard input to tracking algorithms. There are other approaches like Lucas Kanade’s Optical Flow, **Sort based tracking** etc. which are wonderful in their own right based on traditional computer vision methods.

2.4 Prediction Algorithms

Now as we have the Perception module which gets sensor data and does some tasks like, object detection, traffic light detection, traffic light state detection, lane detection,

localization, etc. Then the Behavior Prediction module will get this information and try to predict the future trajectory of other agents in the scene.

2.4.1 Physics-based Models

The process of driving in traffic can at the most fundamental level be seen as a physical process. A model of this physical process can predict the states of a vehicle over time. The first attempts at motion prediction are based on such a physical modeling. The more accurate the model describes the process, the more accurate predictions can be made over time. The models describe the relation between the states of the vehicle at the current time s_t and the next time step s_{t+1} . Simulations of the driving behavior into the future can thus be obtained by iterating the predictions over the model. These models are known as physics-based models or evolution models.

In vehicle motion prediction, simple models are often preferred. When observing vehicles from the ego vehicle's perspective, only a limited number of variables of the subject vehicle can be observed. To this end, physics-based models often are a strong simplification of the vehicle driving process. The physical processes that are modeled consist of dominant dynamic factors, and can be modeled with observable variables of the subject vehicle. Kinematic models are one such category of models that are used often. Rather than modeling the dynamics, the kinematic relations between states such as position, velocity and heading angle over time are used. An important kinematic model is the bicycle model, which takes into account the non-holonomic constraints that apply to vehicles. A variety of similar models are described in [13], such as the Constant Steering Angle and Acceleration (CSAA) model. In these models, the states such as the velocity and position at a next time step can be obtained through the kinematic relations. These states can be iterated over the model to obtain a multiple time-step prediction.

Physics-based models can predict the future only accurately over a small prediction horizon. They are unable to predict variables that are directly related to human input, such as acceleration (throttle) or steering angle. By lack of a model, these variables are assumed to take the constant value of the last observation over the entire trajectory. Alternatively, multiple models with different constant variables can be combined in an Interactive Multiple Model (IMM) filter. The IMM is a much used modeling technique, where a weighted combination of predictions from various physics-based models is used. Other methods such as Switching Kalman Filters or heuristic methods follow a similar approach. These methods have been popular and as of today often are still used to benchmark performances of state-of-the-art approaches. By combining multiple models, more complex driving behavior can be described as a combination of sub trajectories. However, methods to determine when and why the models should be switched for prediction is often lacking.

2.4.2 Maneuver recognition

To obtain better long-term prediction, methods were explored where a prediction is generated from a higher abstraction level. Maneuver recognition approaches try to find an appropriate trajectory with a recognized maneuver, rather than generating a sequence of states based on their previous states. The intuition behind maneuver recognition approaches is that all driving behavior that is observed can be categorized in a set of ma-

maneuver classes. If a maneuver is recognized, a maneuver-specific model can be deployed to get a prediction of the driving behavior. In this way, a part of the human influence of driving behavior can be taken into account by incorporating the maneuvers that are known to drivers in the motion prediction approach.

Recognition is done by classifying an observed trajectory as a maneuver, often by looking at a set of handcrafted features or comparing it to class-specific prototypical trajectories. In the latter case is often focused on a sophisticated feature that can accurately resemble the important aspects of a maneuver, making it straightforward to compare observed trajectories with prototypes, such as in [14]. Once the maneuver is recognized, there are two approaches to generate a prediction. The first simply takes the most resembling prototypical trajectory of the recognized maneuver class. The level of similarity between the observed driving behavior and the prototype can also be used to construct a prediction as a combination of multiple prototypes. The second approach is to use maneuver-specific models per maneuver class to generate a new trajectory. A set of physics-based models can be used with different values for the human-input related variables [15]. The maneuver recognition approach thus combines prior knowledge, e.g. distinguishing between known maneuvers, with an online prediction that is tailored to the specific class.

2.5 Planning and Decision Making

2.5.1 Global Planning

The global planner is responsible for finding the route on the road network from origin to the final destination. The user usually defines the final destination. Global navigation is a well-studied subject, and high performance has become an industry standard for more than a decade. Almost all modern production cars are equipped with navigation systems that utilize GPS and offline maps to plan a global route. Route planning is formulated as finding the point-to-point shortest path in a directed graph, and conventional methods are examined under four categories in [Route Planning in Transportation Networks](#). These are; [goal-directed](#), [separator-based](#), [hierarchical](#) and [bounded-hop techniques](#).

A combination of the above algorithms are popular in state-of-the-art systems. For example, [this research](#) combined a separator with a bounded-hop method and created the Transit Node Routing with Arc Flags (TNR+AF) algorithm. Modern route planners can make a query in milliseconds.

2.5.2 Local Planning

The objective of the local planner is to execute a global plan without failing. In other words, in order to complete its trip, the automated driving system must find trajectories to avoid obstacles and satisfy optimization criteria in the configuration space (Cspace), given a starting and destination point. A detailed local planning review is presented in this [research work](#) where the taxonomy of motion planning was divided into four groups; graph based planners, sampling-based planners, interpolating curve planners and numerical optimization approaches. After a summary of these conventional planners, the emerging deep learning-based planners are the latest state-of-the-art of this section. Table 3 gives a brief summary of local planning methods.

Moreover, for navigating securely in the proposed trajectory, most of the autonomous vehicles make use of occupancy grid maps [16] to avoid obstacles. The basic idea of occupancy grid is to represent the environment in a map with evenly spaced field of binary random variables. Each of these variables represent the presence of an obstacle in the environment.

Table 3. Local Planning Techniques

Approach	Method	Pros and Cons
Graph Search	Dijkstra, A*, State Lattice	Slow and Jerky
Sampling Based	RPP, RRT, RRT*, PRM, Particle Filter	Fast solution but jerky
Curve Interpolation	Clothoids, Polynomials, Bezier, Splines	Smooth but Slow
Numerical Optimization	Num. Non-linear opt., Newton's Method	increases computational cost but improves quality
Deep Learning	FCN, Segmentation network	high imitation performance but no-hard coded safety measures

2.5.3 Decision Making

For many years the go-to approach for handling decision making for intelligent systems were the Finite State Machines (FSM). However, FSMs are prone to to modularity, leading to the introduction of the behavior trees technique as an alternative.

Behavior trees (BT) is a control policy representation that increases modularity, simplifying that way human and machine design. BTs are comprised of a sequence of simple actions that are being executed, given that the preconditions are activated.

For example, in the case of our autonomous vehicle, the behavior tree is fed up with the indication about its battery level, in order to determine whether to interrupt the operation of navigation, to go to a charging station as it is shown above.

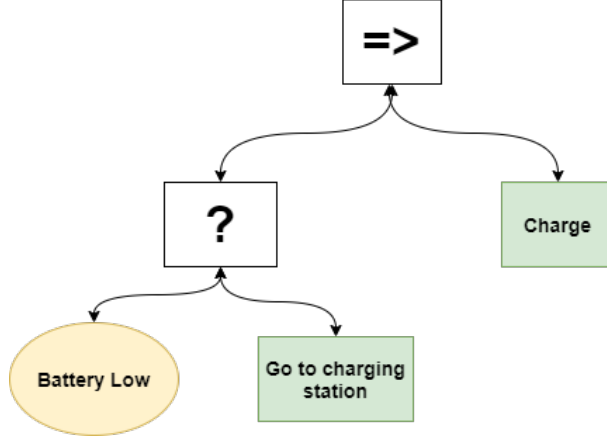


Figure 2: Behavior tree for battery management

2.6 Acting

2.6.1 Control

Control refers to the vehicle's ability to execute the planned actions that have been generated from the planning and decision making process. It is the process of converting intentions into actions. There are three type of controller structures that are mostly used in autonomous vehicles applications [17]:

- Feedback control
- Model Predictive Control
- Trajectory Generation and Tracking

Feedback control compares the measured system response with the desired behavior and compensates accordingly to minimize deviations. The most common form of feedback control is the **Proportional - Integral - Derivative (PID)** controller where the control law is based on the error signal as:

$$u(t) = k_d \dot{e} + k_p e + k_i \int e(t) dt \quad (1)$$

where e is the error signal and the k_p, k_i, k_d are the proportional, integral and derivative gains of the controller respectively. The error signal then it is fed into the state space control.

However feedback control has some delay and coupled response limitations. Some times this is solved with the addition of a feedforward term (two degree of freedom controller), but it requires precise knowledge of the physical model.

Model Predictive Control (MPC) uses system modelling to optimize the control of the vehicle over a given time period. It was developed by combining the performance of optimal control and the robustness of the robust control. MPC is suitable for handling

non-linearities, complicated process models, time delays. Thus it is useful for repeated tasks such as path following. MPC is used for several automotive control applications such as traction control, braking and steering, lane keeping etc. Displaying and explaining the formula behind MPC is beyond the scopes of this document.

Trajectory Generation and Tracking . There are two ways to generate trajectory. Either by optimizing trajectory generation and tracking simultaneously either by decoupling these. The former is time challenging due to the limited processing power. Thus it is not preferred for complex environments. The latter defines two separate problems.

The trajectory generation seeks a control input $u(t)$ that corresponds to the desired state trajectory $x(t)$. This is solved either using a sensor-based approach, which focuses on integrating the perception of the environment without taking much of the dynamics into account, either a dynamic-based approach.

On the other hand, trajectory tracking ensures following the generated trajectory. There are two types of methods for that: geometric and model based methods. The most popular geometric method is the **pure pursuit path tracking algorithm** where the vehicle's steering angle δ can be defined by,

$$\delta(t) = \tan^{-1} \left(\frac{2L \sin(\eta(t))}{kv(t)} \right) \quad (2)$$

The **Stanley method** is another popular geometric steering controller. Compared to the pure pursuit it has better tracking results and performs better at high speed driving. However it is prone to disturbances and may result in oscillations.

Regarding the model-based trajectory tracking, vehicle kinematic models are used at slower speeds. For higher speeds, a kinematic car controller, based on kinematic bicycle model is used [18], or its upgraded chained form [19].

Last, MPC can also be used for the trajectory tracking problem [17] such as its variations of:

- Path Tracking Model Predictive Controller
- Unconstrained MPC with Kinematic Model
- MPC Trajectory Controller with Dynamic Car Model

2.7 End to End Learning

End to end learning is an alternative novel technique that have been developed the last years, that follows a less complex and more intelligent approach than the sequential perception-prediction-planning-acting pipeline. Instead, it outputs motion commands directly from sensory inputs without tracking explicitly how the information is processed in it. The processing of the inputs lies on deep learning. There are three main approaches for end to end learning.

- **Direct supervised deep learning** [20] [21] [22] [23] [24]. This technique imitates the human driver and can be trained offline. However, it has a poor generalization performance and it requires the ground truth for training.

- **Neuroevolution** [25] [26]. Neuroevolution uses evolutionary algorithms to train ANNs. Its advantage is that it has no backpropagation. The drawback is that it requires online interaction and it is still in experimental phase since no real world driving has been achieved yet.
- Similarly, **Deep reinforcement learning** [27] [28] had not been achieved in an urban environment yet. A successive model of this approach is the Deep Q Networks (DQN). It learns the optimum way of driving - instead of imitating the drive - by selecting the set of actions that maximize the cumulative future rewards. DQN also requires online interaction.

2.8 Advanced Capabilities

2.8.1 Parking

Autonomous Vehicles revolution is bringing novel concepts in the transportation industry. AVs may not be required to park anymore since transportation may change from an individual privilege to a mobility as a service (MaaS). However, parking is still a necessary and challenging task for AVs. The most common techniques used for automated parking [29] are:

- Geometric methods like the trajectory planning for parking using clothoids curves [30]
- Three-step methods
- Sampling-based methods like RRT, Hybrid A* and DWA
- Numerical optimization methods by using for example sequential quadratic programming (SQP)
- Deep Reinforcement Learning [31], [32], [33]

The geometric approach has the advantage of fast computation compared to the others. The three step and sampling-based approaches may result in a trajectory with too many cusps. While Numerical optimization methods are very time-consuming.

The most recent development in this section regards deep reinforcement learning which seems be very promising.

3 System Requirements, Architecture and Specifications

3.1 Requirements

- Perception
 - The car shall recognise the traffic signs (turn right/left, go straight, construction area, slipperiness, stop and parking).
 - The car shall recognise the lanes on the road.
 - The car shall recognise people and objects (dynamic or static) in the environment.
 - The car shall generate a local dynamic map of the current traffic situation in the environment.
- Prediction
 - The car shall predict future behaviour of traffic entities in the environment.
 - The car shall predict the course of the track given the lanes and traffic signs.
- Navigation
 - Given the global planner, the car shall follow the lanes on the road to keep itself within the track.
 - The car shall avoid obstacles coming across its way (e.g. to people and objects moving towards the lane).
 - The car should keep a safe distance to other robots in the environment.
- Advanced Behaviour
 - The robot shall be able to find an available spot to park. It could be a parking-labeled spot by default or a free segment on the street alternatively.
 - The robot shall create a local, smooth and safe trajectory planning to park.
 - The robot shall be able to recharge its battery at a charging station if it is needed.

3.2 System Architecture

As the simulation environment of the project is set-up in Morse, we have a *morse* node publishing topics related to different sensors as shown in figure 3. These topics are then subscribed by other nodes to process raw information coming from sensors and carry out odometry estimation, perception, prediction, dynamic mapping, and path planning to achieve a given goal. We intend to develop the Perception, Prediction, Path Planner, and Dynamic Mapping packages while improving the already existing packages provided in ROS.

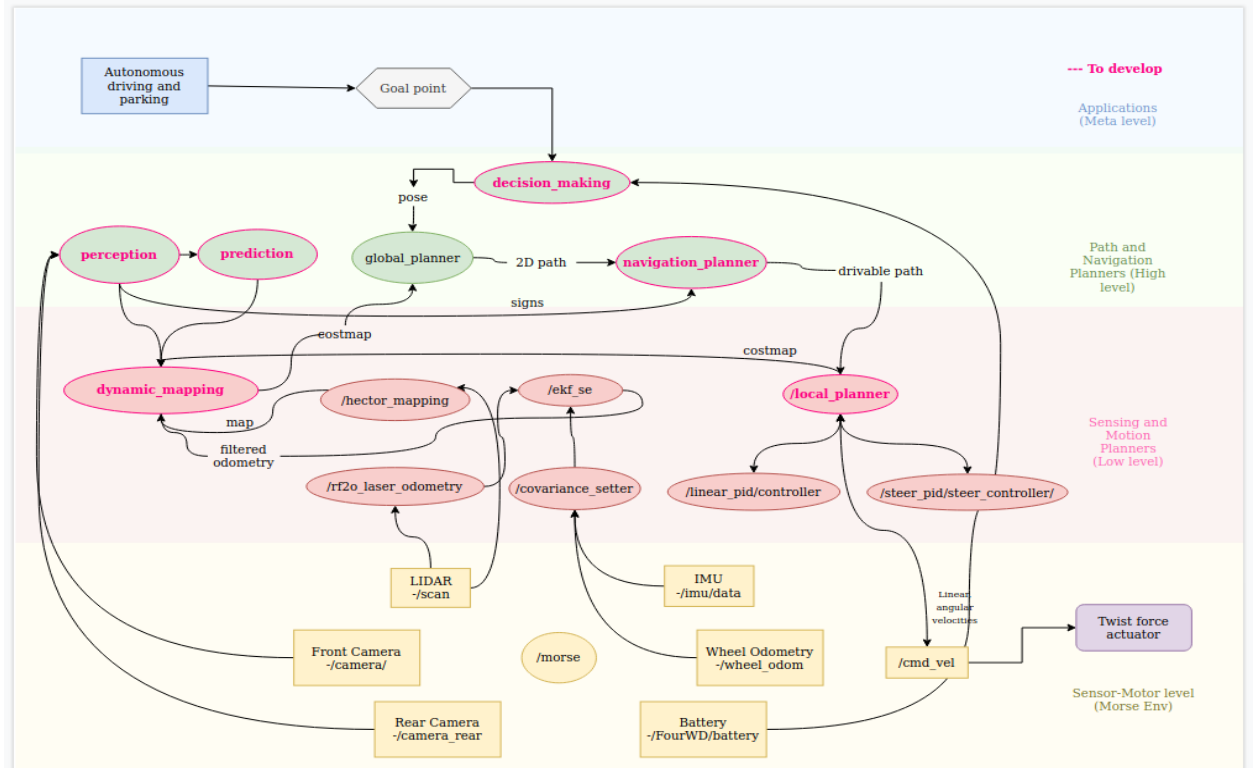


Figure 3: System Architecture and Flow Diagram

3.3 System Specifications

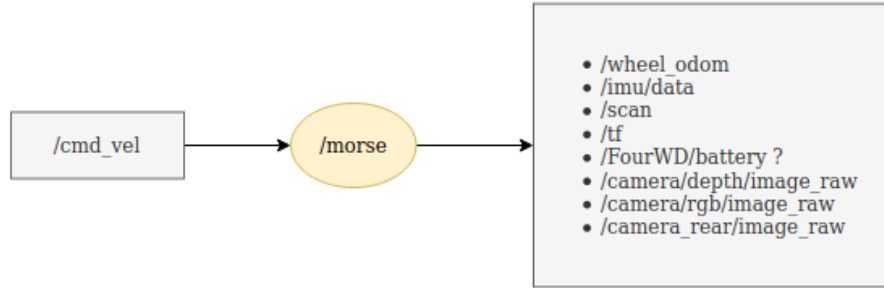


Figure 4: Subscribed and published rostopics for `/morse` node since the car and environment is setup in morse.

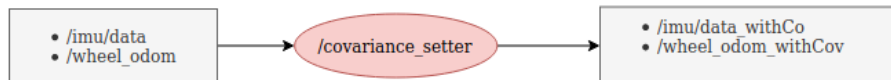


Figure 5: Subscribed and published rostopics by `/covariance_setter` node. This node fixed the missing covariance values in the odometry sensor readings.

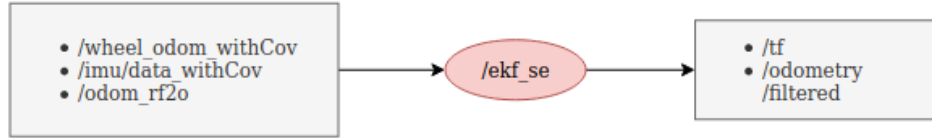


Figure 6: Subscribed and published rostopics by `/ekf_se` node. This node estimates robot position.

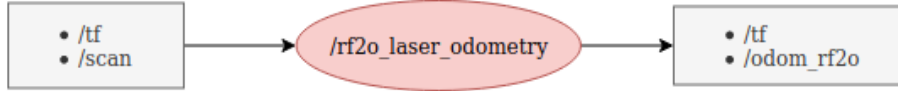


Figure 7: A transformation node, converting the readings from the laser scans to odometry values to feed the hector mapping.

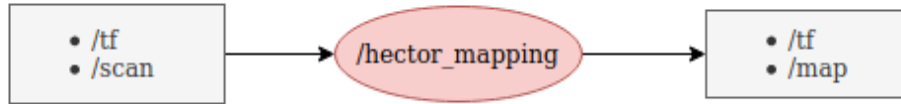


Figure 8: Hector mapping taking the lidar scan readings and publishing the updated map.



Figure 9: Inputs and outputs for perception node. We intend to use YOLO deep learning architecture for object detection and classification.

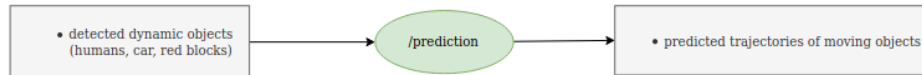


Figure 10: Inputs and outputs for prediction node. Trajector++ method will be used for object tracking.

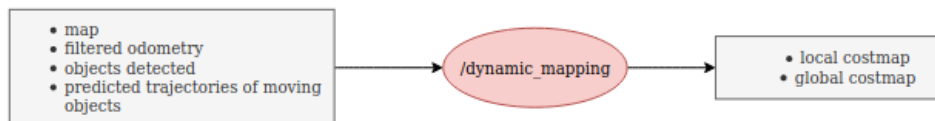


Figure 11: Dynamic mapping producing costmaps after projecting the environment perceived and predicted on a costgrid.

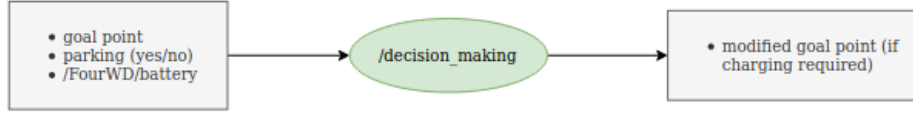


Figure 12: Decision making node taking into account the current battery status and modifying the given goal points as to accommodate charging stations in the path if required. We intend to use **behaviour trees** for implementation.

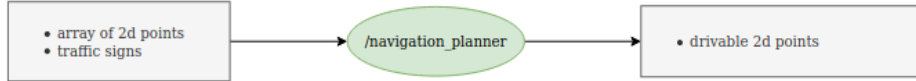


Figure 13: Navigation node for guiding the car to follow traffic signs.

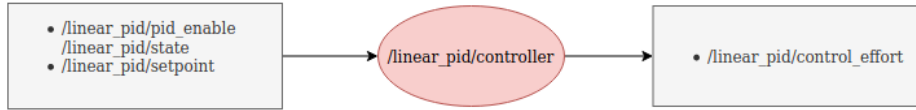


Figure 14: Subscribed and published rostopics by the *linear_pid* controller node.

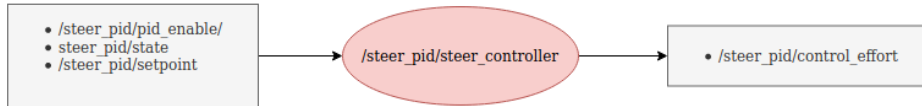


Figure 15: Subscribed and published rostopics by the *steer_pid* controller node.



Figure 16: Local planner taking in the local costmap and pose to follow it closely using inputs from pid controller. The velocities are published on */cmd_vel* topic.

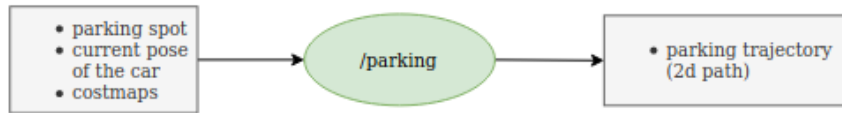
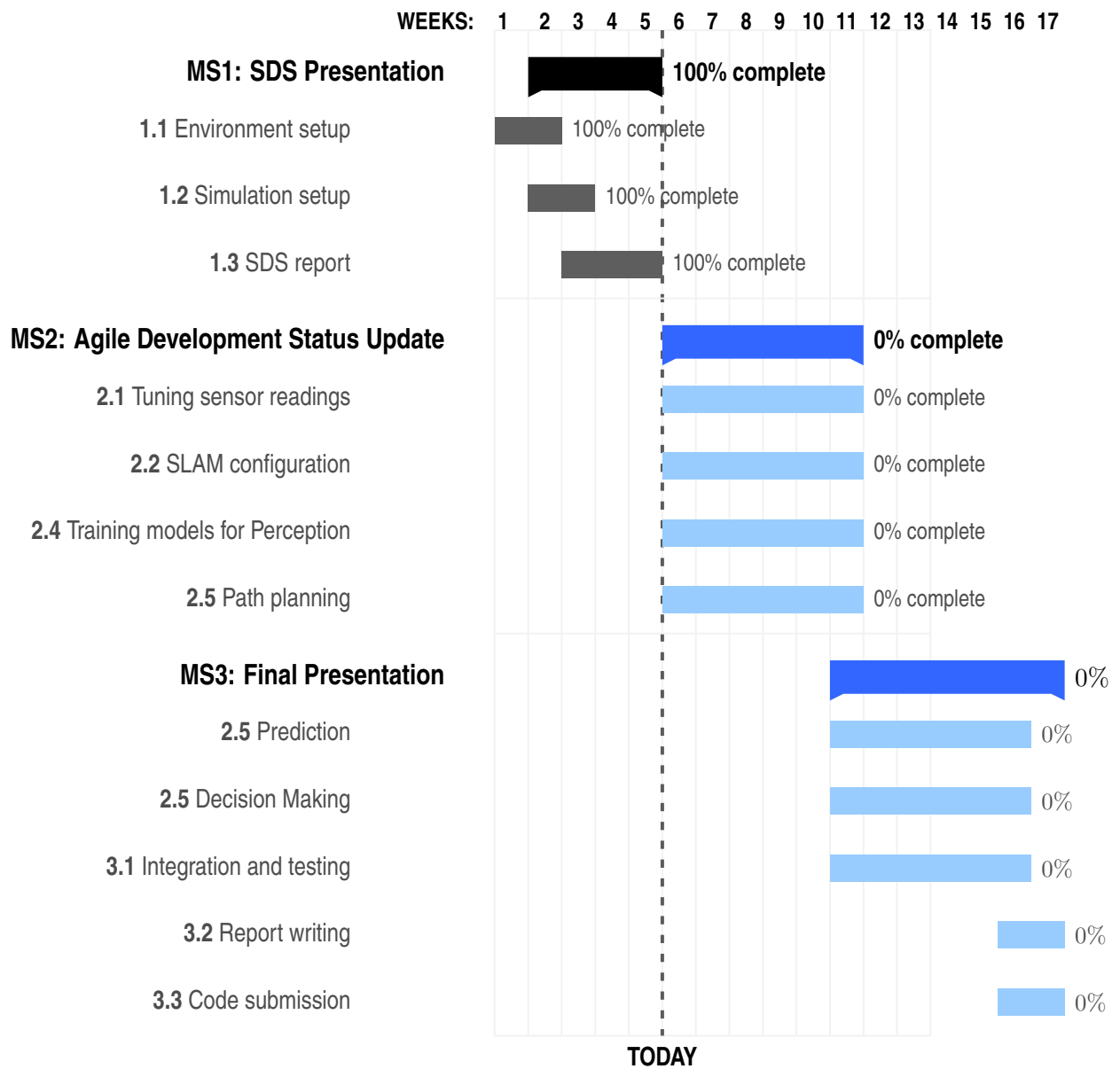


Figure 17: The parking node will use reinforcement learning to estimate parking trajectory given the parking sport, costmaps and current pose of the car.

4 Timeline



References

- [1] A. Nikiforuk, "The big shift last time: From horse dung to car smog," Mar 2013. [Online]. Available: <https://thetyee.ca/News/2013/03/06/Horse-Dung-Big-Shift/>
- [2] "Become a Self-Driving Car Engineer." [Online]. Available: <https://www.udacity.com/course/self-driving-car-engineer-nanodegree--nd013>
- [3] B. Geenen, "Autonomous vehicle technology report," 2020. [Online]. Available: <https://wevolver-project-images.s3-us-west-1.amazonaws.com/Wevolver+2020+Autonomous+Vehicle+Technology+Report.pdf>
- [4] G. Bresson, Z. Alsayed, L. Yu, and S. Glaser, "Simultaneous localization and mapping: A survey of current trends in autonomous driving," *IEEE Transactions on Intelligent Vehicles*, 2017. [Online]. Available: <https://ieeexplore.ieee.org/document/8025618>
- [5] R. Mur-Artal and J. D. Tardós, "Orb-slam2: an open-source slam system for monocular, stereo and rgb-d cameras," 2016. [Online]. Available: [arXiv:1610.06475](https://arxiv.org/abs/1610.06475)
- [6] J. Tang, L. Ericson, J. Folkesson, and P. Jensfelt, "Gcnv2: Efficient correspondence prediction for real-time slam," 2019. [Online]. Available: [arXiv:1902.11046v3](https://arxiv.org/abs/1902.11046)
- [7] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, *A Survey of Autonomous Driving: Common Practices and Emerging Technologies*, Jun. 2019.
- [8] L. Martin, "The state of 3D object detection," Aug. 2020. [Online]. Available: <https://towardsdatascience.com/the-state-of-3d-object-detection-f65a385f67a8>
- [9] "Lane Detection: Brief History and the Road Ahead | by Wajahat Kazmi | keeptruckin-eng | Medium." [Online]. Available: <https://medium.com/keeptruckin-eng/lane-detection-a-quick-review-and-the-way-forward-dd7a43353da>
- [10] S. Lee, J. Kim, J. S. Yoon, S. Shin, O. Bailo, N. Kim, T.-H. Lee, H. S. Hong, S.-H. Han, and I. S. Kweon, "VPGNet: Vanishing Point Guided Network for Lane and Road Marking Detection and Recognition," *arXiv:1710.06288 [cs]*, Oct. 2017, arXiv: 1710.06288. [Online]. Available: <http://arxiv.org/abs/1710.06288>
- [11] D. Shah, "Object Tracking," Apr. 2020. [Online]. Available: <https://medium.com/visionwizard/object-tracking-675d7a33e687>
- [12] P. Bergmann, T. Meinhardt, and L. Leal-Taixe, "Tracking without bells and whistles," *arXiv:1903.05625 [cs]*, Aug. 2019, arXiv: 1903.05625. [Online]. Available: <http://arxiv.org/abs/1903.05625>
- [13] S. Lefèvre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles," *ROBOMECH Journal*, 2014. [Online]. Available: <https://robomechjournal.springeropen.com/articles/10.1186/s40648-014-0001-z>

- [14] C. Hermes, C. Wöhler, K. Schenk, and F. Kummert, "Long-term vehicle motion prediction," *IEEE Intelligent Vehicles Symposium*, pp. 652–657, 2009. [Online]. Available: <http://hausmilbe.net/Publications/files/Hermes09.pdf>
- [15] —, "Vehicle trajectory prediction based on motion model and maneuver recognition," *International Conference on Intelligent Robots and Systems (IROS 2013)*, pp. 4363–4369, 2013. [Online]. Available: <https://hal.archives-ouvertes.fr/hal-00881100/document>
- [16] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, vol. 22, no. 6, pp. 46–57, Jun. 1989. [Online]. Available: <http://ieeexplore.ieee.org/document/30720/>
- [17] S. Pendleton, H. Andersen, X. Du, X. Shen, M. Meghjani, Y. Eng, D. Rus, and M. Ang, "Perception, Planning, Control, and Coordination for Autonomous Vehicles," *Machines*, vol. 5, no. 1, p. 6, Feb. 2017. [Online]. Available: <http://www.mdpi.com/2075-1702/5/1/6>
- [18] A. De Luca, G. Oriolo, and C. Samson, "Feedback control of a nonholonomic car-like robot," in *Robot Motion Planning and Control*, M. Thoma and J. P. Laumond, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, vol. 229, pp. 171–253, series Title: Lecture Notes in Control and Information Sciences. [Online]. Available: <http://link.springer.com/10.1007/BFb0036073>
- [19] R. Murray and S. Sastry, "Steering nonholonomic systems in chained form," in *[1991] Proceedings of the 30th IEEE Conference on Decision and Control*. Brighton, UK: IEEE, 1991, pp. 1121–1126. [Online]. Available: <http://ieeexplore.ieee.org/document/261508/>
- [20] C. Chen, A. Seff, A. Kornhauser, and J. Xiao, "DeepDriving: Learning Affordance for Direct Perception in Autonomous Driving," p. 9.
- [21] D. A. Pomerleau, "ALVINN: An Autonomous Land Vehicle in a Neural Network," p. 9.
- [22] Y. LeCun, U. Muller, J. Ben, and E. Cosatto, "Off-Road Obstacle Avoidance through End-to-End Learning," p. 8.
- [23] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, "End to End Learning for Self-Driving Cars," *arXiv:1604.07316 [cs]*, Apr. 2016, arXiv: 1604.07316. [Online]. Available: <http://arxiv.org/abs/1604.07316>
- [24] H. Xu, Y. Gao, F. Yu, and T. Darrell, "End-to-end Learning of Driving Models from Large-scale Video Datasets," *arXiv:1612.01079 [cs]*, Jul. 2017, arXiv: 1612.01079. [Online]. Available: <http://arxiv.org/abs/1612.01079>
- [25] "Evolution of an artificial neural network based autonomous land vehicle controller - IEEE Journals & Magazine." [Online]. Available: <https://ieeexplore.ieee.org/document/499795/>

- [26] J. Koutník, G. Cuccu, J. Schmidhuber, and F. Gomez, “Evolving large-scale neural networks for vision-based reinforcement learning,” in *Proceeding of the fifteenth annual conference on Genetic and evolutionary computation conference - GECCO '13*. Amsterdam, The Netherlands: ACM Press, 2013, p. 1061. [Online]. Available: <http://dl.acm.org/citation.cfm?doid=2463372.2463509>
- [27] A. E. Sallab, M. Abdou, E. Perot, and S. Yogamani, “Deep Reinforcement Learning framework for Autonomous Driving,” *Electronic Imaging*, vol. 2017, no. 19, pp. 70–76, Jan. 2017, arXiv: 1704.02532. [Online]. Available: <http://arxiv.org/abs/1704.02532>
- [28] A. Kendall, J. Hawke, D. Janz, P. Mazur, D. Reda, J.-M. Allen, V.-D. Lam, A. Bewley, and A. Shah, “Learning to Drive in a Day,” *arXiv:1807.00412 [cs, stat]*, Sep. 2018, arXiv: 1807.00412. [Online]. Available: <http://arxiv.org/abs/1807.00412>
- [29] L. Lin and J. J. Zhu, “PATH PLANNING FOR AUTONOMOUS CAR PARKING,” p. 11, 2018.
- [30] H. Fuji, J. Xiang, Y. Tazaki, B. Levedahl, and T. Suzuki, “Trajectory planning for automated parking using multi-resolution state roadmap considering non-holonomic constraints,” in *2014 IEEE Intelligent Vehicles Symposium Proceedings*. MI, USA: IEEE, Jun. 2014, pp. 407–413. [Online]. Available: <http://ieeexplore.ieee.org/document/6856433/>
- [31] A. Folkers, M. Rick, and C. Büskens, “Controlling an Autonomous Vehicle with Deep Reinforcement Learning,” *2019 IEEE Intelligent Vehicles Symposium (IV)*, pp. 2025–2031, Jun. 2019, arXiv: 1909.12153. [Online]. Available: <http://arxiv.org/abs/1909.12153>
- [32] M. Kaushik and K. M. Krishna, “Learning Driving Behaviors for Automated Cars in Unstructured Environments,” in *Computer Vision – ECCV 2018 Workshops*, L. Leal-Taixé and S. Roth, Eds. Cham: Springer International Publishing, 2019, vol. 11133, pp. 583–599, series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/978-3-030-11021-5_36
- [33] P. Zhang, L. Xiong, Z. Yu, P. Fang, S. Yan, J. Yao, and Y. Zhou, “Reinforcement Learning-Based End-to-End Parking for Automatic Parking System,” *Sensors*, vol. 19, no. 18, p. 3996, Sep. 2019. [Online]. Available: <https://www.mdpi.com/1424-8220/19/18/3996>